



School of Future Tech

Case Study Report

REACT JS

Problem Statement No. - 190

**Title - Device Screen Orientation & Motion
Calibration Sandbox**

Name - Stuti Saxena

Roll number - 150096725024

Batch - Larry Page

GitHub -

<https://github.com/stutisaxena44/Motion-Calibration-System>

Live Link -

<https://device-motion-calibration-system-re.vercel.app/>

Index

- 1. Problem Statement**
- 2. Tech Stack**
- 3. Component Architecture**
- 4. Features**
- 5. Screenshots**
- 6. Conclusion**

Problem Statement

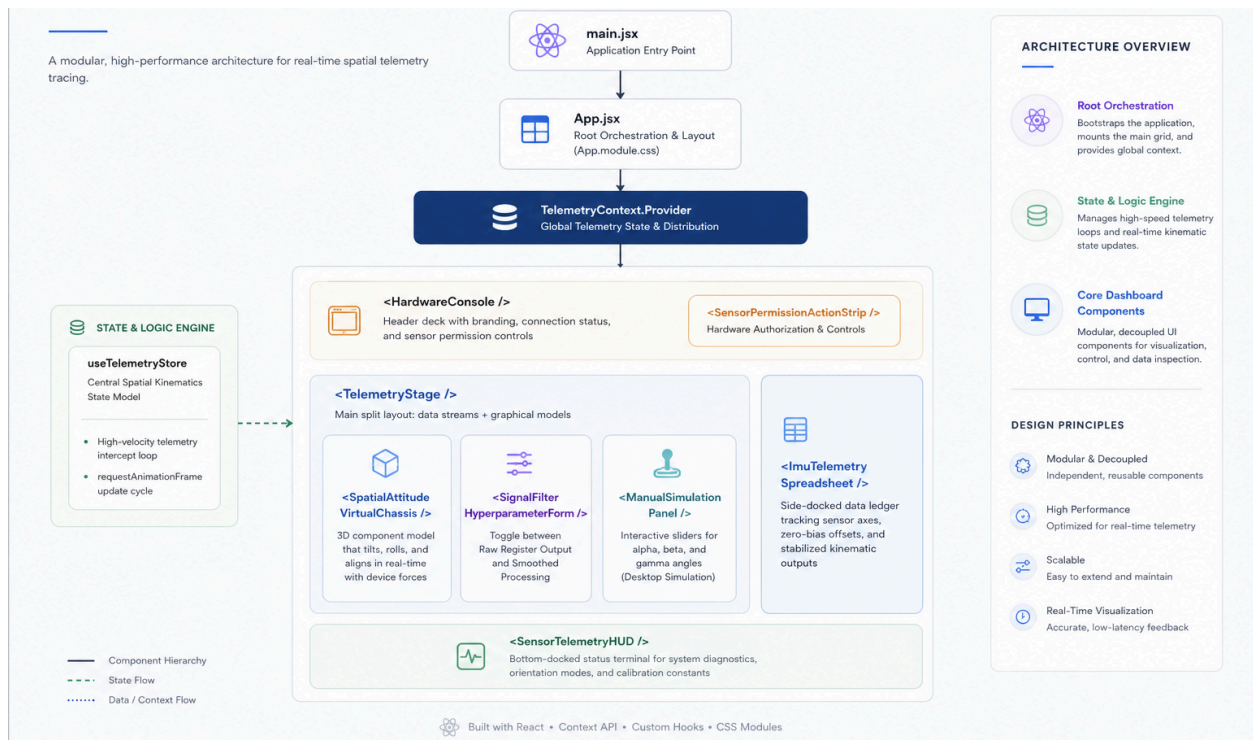
Modern mobile devices contain highly sensitive MEMS (Micro-Electro-Mechanical Systems) hardware, including tri-axial accelerometers and solid-state gyroscopes. Translating their raw, noisy hardware telemetry streams into stable spatial coordinate vectors requires processing real-time orientation matrices, handling angular coordinate singularities, and applying low-pass signal filtration.

This project solves this challenge by delivering a client-side device-motion hardware interceptor, mathematical filtering matrix, and high-frequency 3D vector visualizer built entirely in JavaScript. The application serves as an interactive mobile telemetry laboratory that captures hardware angular velocities, maps spatial tilt vectors, and runs real-time calibration routines with zero network latency and complete data privacy.

Tech Stack

- **Frontend Framework:** React 19 (bootstrapped with Vite)
- **Styling:** CSS Modules (App.module.css) combined with standard CSS variables and inline hardware-accelerated 3D CSS transforms.
- **State Management:** React Context API (TelemetryContext), combined with useReducer for structured application states and useRef for high-frequency volatile motion buffers.
- **Hardware Integration:** Native Web APIs (DeviceOrientationEvent and DeviceMotionEvent).
- **Local Persistence:** Browser localStorage API.

Component Architecture



The application is structured into a highly decoupled, modular architecture within the **src** directory to guarantee high-performance real-time data tracing:

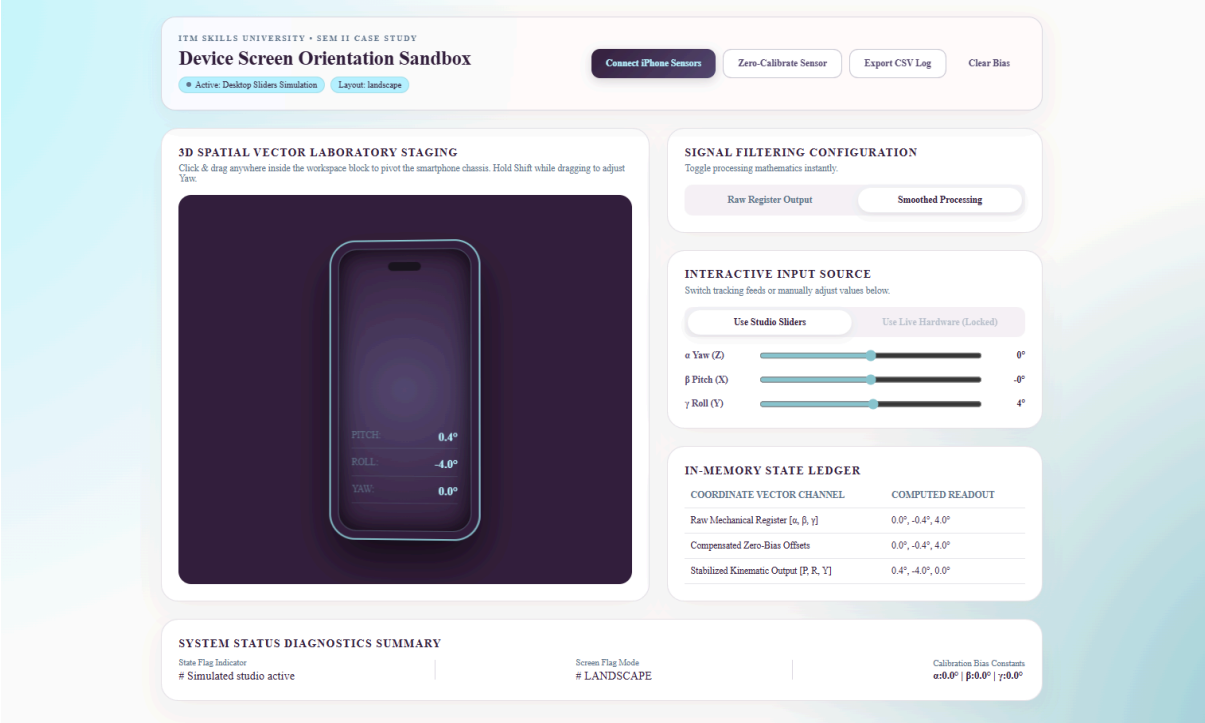
- **Root Orchestration (main.jsx & App.jsx):** Bootstraps the application, mounts the main grid (*App.module.css*), and wraps the application inside the `TelemetryContext.Provider`.
 - **State & Logic Engine:** `useTelemetryStore`: A custom hook functioning as the central spatial kinematics state model. It manages the high-velocity telemetry intercept loop and processes `requestAnimationFrame` updates.
 - **Core Dashboard Components:**
 - **<HardwareConsole />**: The primary header deck housing the project branding, connection source readouts, and the **<SensorPermissionActionStrip />** which handles hardware authorization.
 - **<TelemetryStage />**: The main split layout orchestration component that pairs raw numeric data streams with graphical physics models.
 - **<SpatialAttitudeVirtualChassis />**: A 3D component model container that tilts, rolls, and aligns in real-time, matching the physical forces of the device.
 - **<SignalFilterHyperparameterForm />**: Form controls allowing engineers to toggle between Raw Register Output and Smoothed Processing.
 - **<ManualSimulationPanel />**: An interactive fallback panel utilizing sliders for desktop simulation of alpha, beta, and gamma angles.
 - **<ImuTelemetrySpreadsheet />**: A side-docked data ledger tracking logging structural values for sensor axes, zero-bias offsets, and stabilized kinematic outputs.
 - **<SensorTelemetryHUD />**: A bottom-docked status terminal documenting system diagnostics, orientation flag modes, and calibration constants.
-

Features

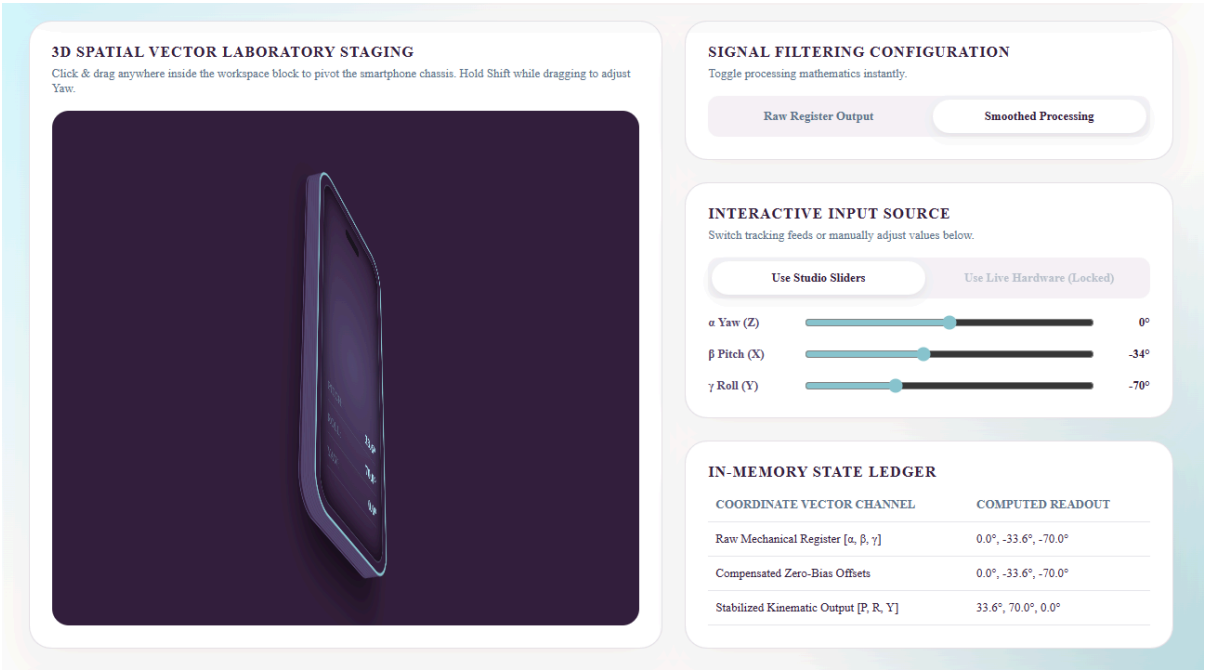
1. **High-Frequency Telemetry Intercept Engine:** The application hooks directly into the browser's native API boundaries, handling high-velocity sensor streams ticking up to 60 times per second. By decoupling React state from the animation frame loop (`requestAnimationFrame`), it modifies DOM matrix transforms directly, avoiding UI lag.
2. **Signal Noise Complementary Filter:** To resolve gyroscope drift and accelerometer vibration noise, the sandbox runs a mathematical filtering pipeline. It blends inputs via a tunable algorithm to eliminate jitter and output a smooth vector stream.
3. **Dynamic Sensor Bias Calibration:** Includes an active "Zero-Calibration Routine". By sampling IMU streams over a brief frame window, it computes the arithmetic mean offset and saves these as negative vector constants to neutralize structural hardware inaccuracies.
4. **Hardware-Accelerated 3D Chassis Matrix:** A visually rich, multi-layered 3D virtual smartphone that perfectly mimics the physical hardware's orientation (Pitch, Roll, Yaw) using pure CSS 3D transforms connected to the active data stream.
5. **Interactive Desktop Simulation Studio:** For testing on hardware without physical sensors, users can utilize the interactive multi-axis sliders, or click and drag across the viewport to manually inject telemetry data directly into the simulation ledger.
6. **In-Memory Ledger & CSV Export:** Operates strictly as an in-memory spatial coordinate ledger without a database. Captured telemetry streams can be downloaded instantly to the user's machine as a formatted CSV data log.
7. **LocalStorage Memory Integration:** Custom sensor calibration bias profiles and dashboard layout selections are saved directly to `localStorage` to ensure perfect reloading between sessions.

Screenshots

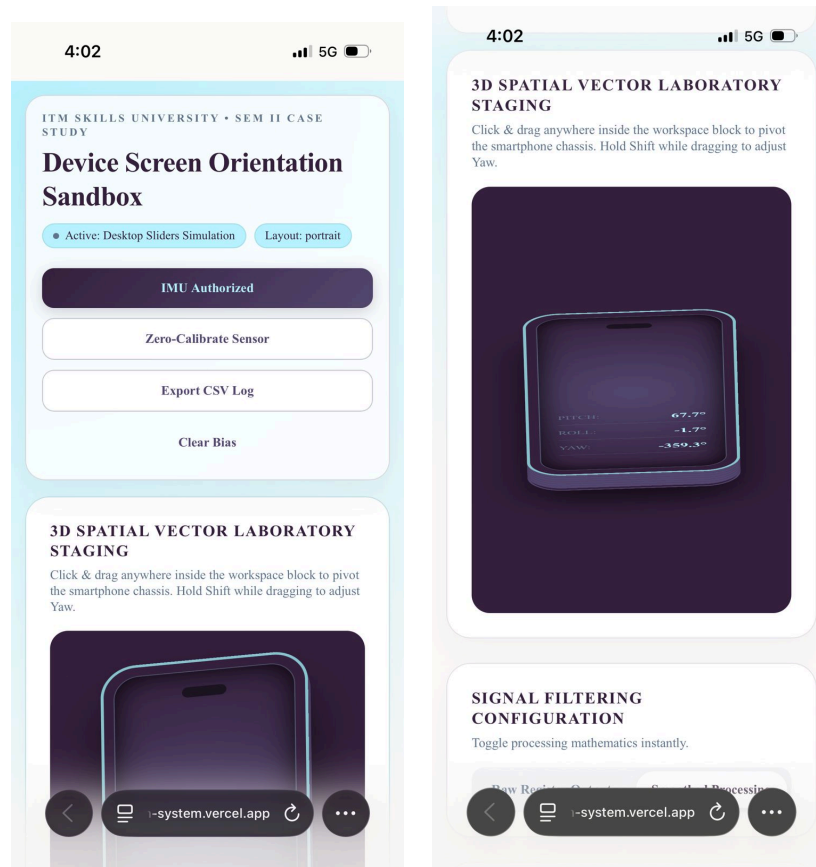
1. The Primary Diagnostic Dashboard



2. The 3D Spatial Virtual Chassis in Action



3. Hardware Access on Mobile (iPhone/iOS)



Conclusion

The Device Screen Orientation & Motion Calibration Sandbox effectively bridges the gap between hardware engineering and modern web development. By leveraging advanced React design patterns - such as decoupling volatile telemetry buffers from standard Context reconciliation - the application hits flawless 60 FPS performance entirely on the client side. The implementation of custom complementary filtering mathematics, a real-time 3D coordinate visualizer, and strict local storage management results in a premium, highly specialized laboratory tool that processes complex hardware signals with zero network latency.